CHƯƠNG 4:
TÍNH TOÁN BẤT ĐỒNG BỘ TRONG PYTHON

Phần I: Tổng quan các Mô hình Tính toán Nền tảng
(Tuần tự • Đồng thời • Song song • Bất đồng bộ)

Giảng viên hướng dẫn: ThS. Lê Anh Nhã Uyên (lanuyen@ctuet.edu.vn)
Khoa Công nghệ Thông tin — Trường Đại học Kỹ thuật - Công nghệ Cần Thơ

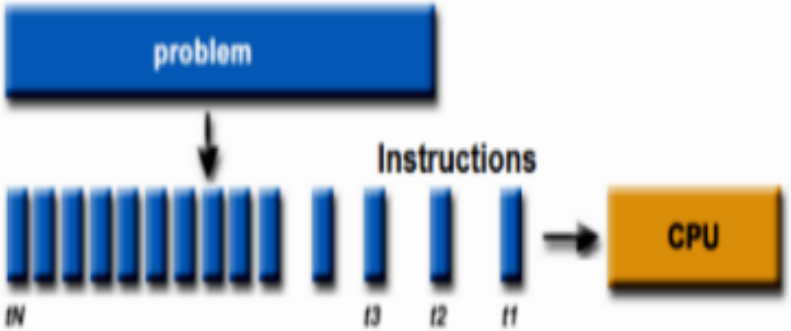
GIỚI THIỆU

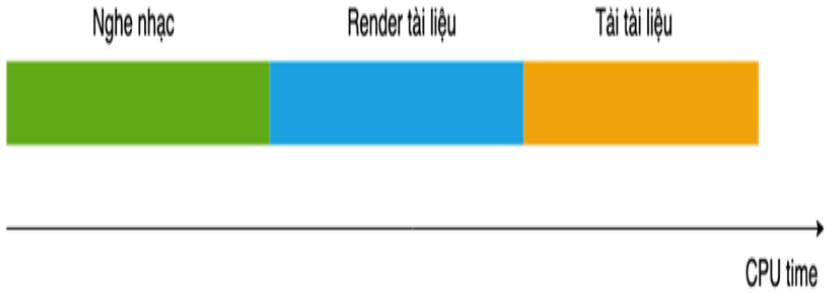
➢ Xử lý tuần tự

□ Xử lý 1 tác vụ (task) trong một khoảng thời gian;

□ Các task sẽ được thực thi theo thứ tự (nối tiếp nhau);

□ Các chỉ thị lệnh được thực thi trên cùng 1 CPU; tại một thời điểm chỉ có 1 chỉ thị lệnh được thực thi.





GIỚI THIỆU

➢ Xử lý đồng thời

□ HĐH quản lý chương trình, mỗi chương trình ~ process (tiến trình) → cấp process id (PID) để HĐH quản lý.

□ CPU core chia các task của process thành các task nhỏ hơn + sắp xếp xen kẽ nhau → tận dụng thời gian rảnh của task này đi thực hiện task khác.

□ → Máy tính xử lý nhiều việc cùng lúc tại cùng thời điểm, tuy nhiên, bên dưới CPU core chỉ có thể thực thi một task nhỏ trong task lớn tại thời điểm đó.



GIỚI THIỆU

➢ Xử lý đồng thời (tiếp theo)

□ Phân chia, điều phối nhiều tác vụ (task) khác nhau trong cùng một khoảng thời gian.

□ Tại 1 thời điểm, chỉ xử lý 1 task.

□ Tối ưu hóa cấu trúc chương trình, giúp hệ thống luôn phản hồi và không bị treo khi chờ đợi.



GIỚI THIỆU

➢ Xử lý song song

- □ Nhiều task khác nhau được xử lý trong cùng 1 thời điểm;
- □ Các task hoàn toàn độc lập nhau.
- □ Chỉ có thể thực hiện trên máy có số nhân (core) lớn hơn 1.



GIỚI THIỆU

➢ Đồng thời và song song

□ Trên thực tế, trên mỗi nhân của CPU vẫn xảy ra quá trình xử lý đồng thời, với điều kiện là tại một thời điểm không xảy ra việc xử lý cùng một task trên hai nhân CPU khác nhau.

□ Đơn nhân (Single-core): Điều phối xen kẽ thời gian giữa các tác vụ (Concurrency).

□ Đa nhân (Multi-core): Thực thi song song thực sự trên các nhân phần cứng độc lập (Parallelism).



GIỚI THIỆU

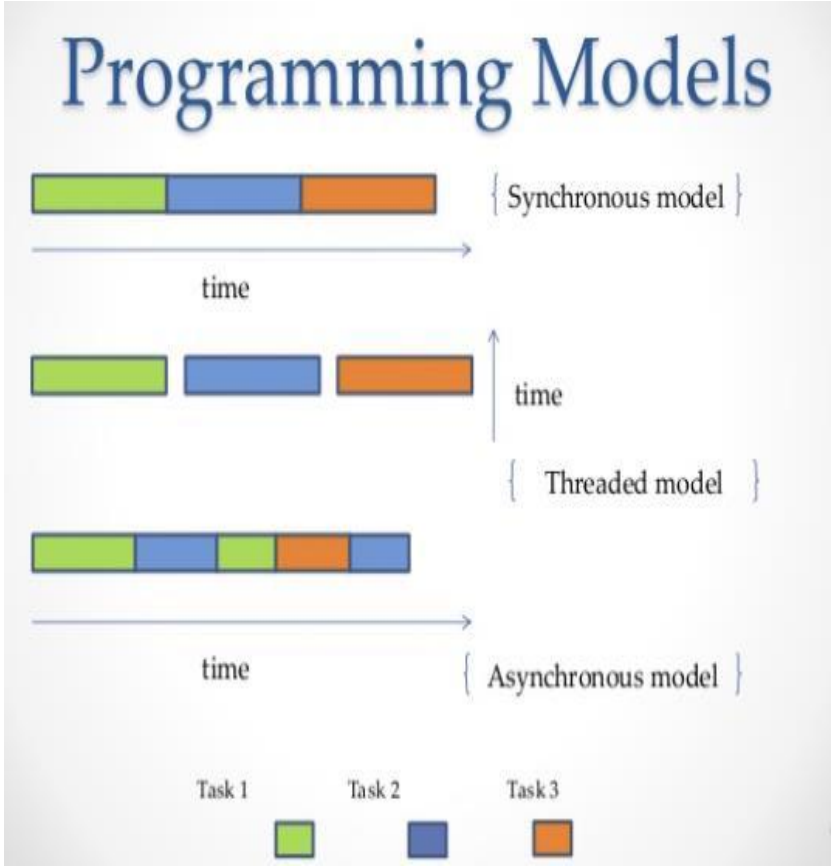
➢ Bất đồng bộ

□ Cho phép các task được xử lý không theo thứ tự (có thể chuyển sang task khác trước khi task trước đó hoàn thành).

□ Có thể giải quyết nhiều yêu cầu đồng thời.

□ Hoàn thành trong khoảng thời gian ngắn hơn.

□ Ví dụ minh họa trong bài giảng: Đi thi trắc nghiệm, Bãi giữ xe...



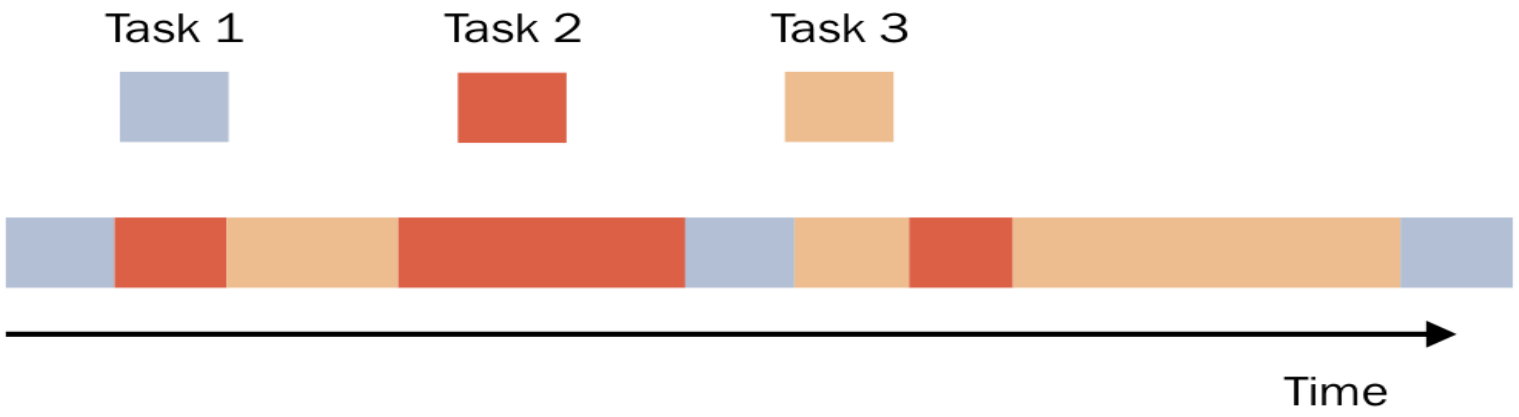
GIỚI THIỆU

➢ Mô hình bất đồng bộ thực thi đồng thời

□ Trong một khoảng thời gian, các task thực hiện xen kẽ nhau;

□ Nằm trong một luồng điều khiển duy nhất → khi 1 task đang thực thi thì các task khác không thực thi (việc thực thi của 1 task có thể bị tạm dừng và sau đó tiếp tục lại);

□ Sử dụng một luồng điều khiển chính duy nhất, cả trong hệ thống đơn bộ xử lý (uniprocessor systems) và hệ thống đa bộ xử lý (multiprocessor systems).



GIỚI THIỆU

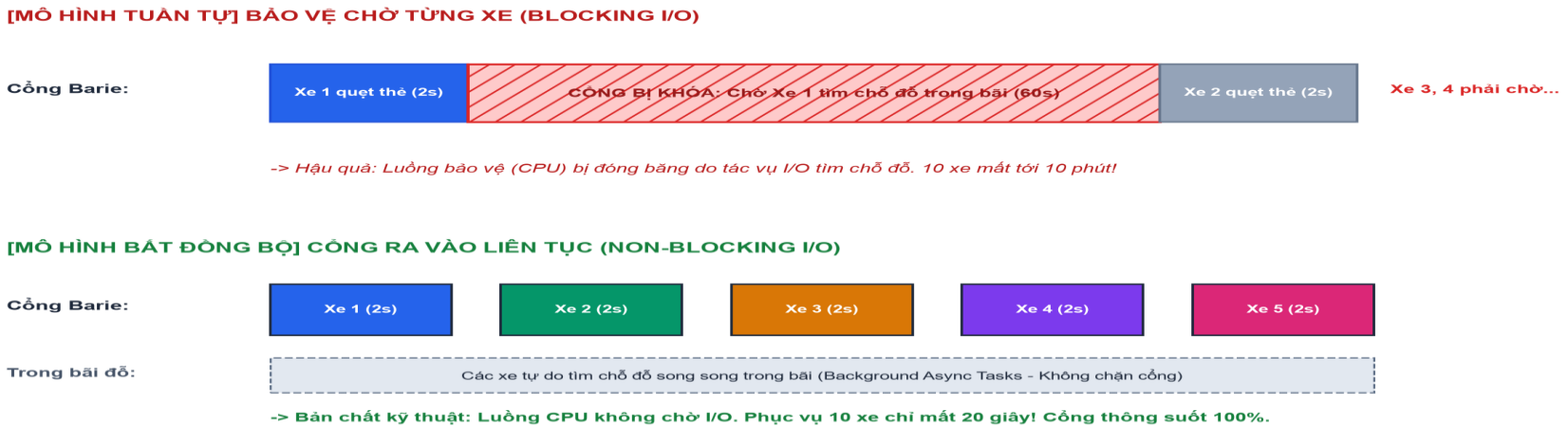
➢ Ví dụ dẫn chứng 1: Bãi giữ xe thông minh



Phân tích cơ chế kỹ thuật từ ví dụ bãi giữ xe:

• Tuần tự (Blocking I/O): Bảo vệ quẹt thẻ cho Xe 1 → Đứng chờ Xe 1 tìm chỗ đỗ xong mới đón Xe 2. Toàn bộ cổng vào bị phong tỏa (CPU Stalled), ùn tắc nghiêm trọng.

• Bất đồng bộ (Non-blocking I/O): Bảo vệ quẹt thẻ 2s → Xe 1 vào tự tìm chỗ ngầm (Async Task) → Đón ngay Xe 2, 3... Cổng thông suốt 100%, phục vụ 10 xe chỉ mất 20 giây!



GIỚI THIỆU

➢ Ví dụ dẫn chứng 2: Làm bài thi trắc nghiệm



Phân tích cơ chế kỹ thuật từ ví dụ đi thi:

• Tuần tự (Blocking): Làm Câu 1 → Câu 2 → Gặp Câu 3 khó ngồi suy nghĩ 40 phút → Hết giờ mà chưa kịp đọc các câu sau! Kết quả bị nghẽn, chỉ đạt 2/10 điểm.

• Bất đồng bộ (Yield & Resume): Gặp Câu 3 khó → Tạm gác lại (Yield) → Nhường não bộ giải quyết ngay Câu 4 đến 8 dễ → Quay lại giải tiếp Câu 3 (Resume) khi có kết quả!

[CHIẾN LƯỢC TUẦN TỰ] BỊ KẸT CỨNG TẠI CÂU KHÓ (BLOCKING)

Tiến trình thi: Câu 1 (1p) | Câu 2 (1p) | CÂU 3 QUÁ KHÓ: Ngồi nghĩ 40 phút (CPU Stalled / Blocked) | HẾT GIỜ! Bỏ trắng C4-C10

-> Hậu quả: Toàn bộ bài thi bị đóng băng tại 1 câu nghẽn. Kết quả chỉ đạt 2/10 điểm!

[CHIẾN LƯỢC BẤT ĐỒNG BỘ] TẠM GÁC CÂU KHÓ (YIELD & RESUME)

Tiến trình thi: Câu 1 | Câu 2 | C3: YIELD (Tạm gác) | Làm nhanh Câu 4, 5, 6, 7, 8 (Dễ) | C3: RESUME (Quay lại giải tiếp)

-> Bản chất kỹ thuật: Cơ chế Hợp tác đa nhiệm (Cooperative Multitasking). Không để tác vụ chờ làm nghẽn toàn bộ hệ thống!

GIỚI THIỆU

➢ Ứng dụng thực tế: Máy chủ Web & I/O Bất đồng bộ



Tại sao công nghệ hiện đại chuyển sang Bất đồng bộ?

• Máy chủ Đồng bộ (Synchronous Server): Mỗi kết nối chiếm 1 Thread. Khi truy vấn Database hoặc đọc file (I/O chờ), Thread bị treo cứng → Cạn kiệt tài nguyên (C10K Problem).

• Máy chủ Bất đồng bộ (Async Event Loop): 1 luồng duy nhất kết hợp cơ chế Non-blocking I/O của HĐH (epoll/kqueue) → Tiếp nhận hàng chục ngàn kết nối đồng thời (FastAPI, Nginx, Node.js).

[MÁY CHỦ ĐỒNG BỘ - SYNCHRONOUS I/O] 1 THREAD / 1 REQUEST (BLOCKING)

Worker Thread: | Req 1: Xử lý | CHỜ DATABASE / ĐỌC FILE (200ms) - Luồng bị treo cứng | Trả về Req 1 | Req 2, 3 bị từ chối / sập!

-> Hạn chế: Khi có 10,000 người truy cập, hệ thống cạn kiệt tài nguyên (C10K Problem), thời gian phản hồi cực chậm.

[MÁY CHỦ BẤT ĐỒNG BỘ - ASYNC EVENT LOOP] NON-BLOCKING I/O (PYTHON ASYNCIO)

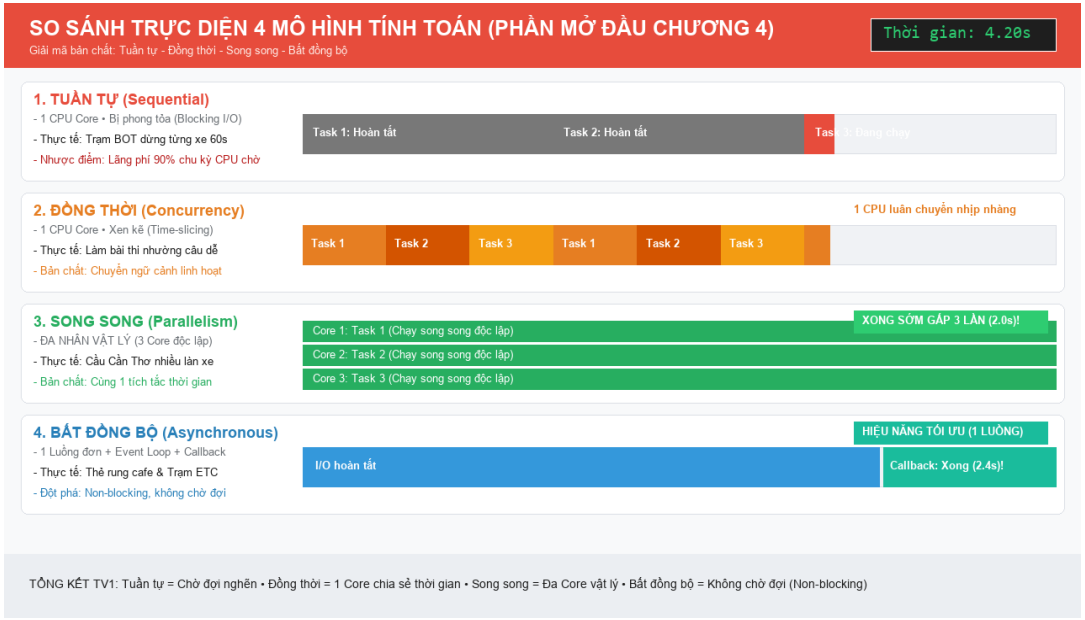
Event Loop: | Req 1 | Req 2 | Req 3 | Req 4 | Req 5 | Callbacks...

Tác vụ I/O HĐH: | Hệ điều hành xử lý I/O ngầm (epoll / kqueue) -> Báo về Event Loop khi có dữ liệu (Non-blocking)

-> Ứng dụng thực tế: Cơ chế của FastAPI, Node.js, Nginx. 1 luồng duy nhất xử lý hàng chục ngàn kết nối đồng thời!

GIỚI THIỆU

➢ Mô phỏng trực quan: So sánh 4 mô hình tính toán



Bản chất 4 mô hình qua mô phỏng:

- Tuần tự: Tác vụ sau phải chờ tác vụ trước, vi xử lý bị nghẽn (Blocking), mất 6.0 giây.
- Đồng thời: 1 CPU chia sẻ thời gian (Time-slicing), chuyển đổi linh hoạt giữa các tác vụ.
- Song song: 3 Core độc lập chạy cùng lúc, hoàn thành sớm gấp 3 lần (chỉ 2.0 giây).
- Bất đồng bộ: 1 Luồng + Event Loop, giải phóng CPU khi chờ I/O, hoàn thành nhanh (2.4 giây).

• Giá trị cốt lõi: Mô phỏng trực quan giúp người học nắm vững bản chất kỹ thuật và sự khác biệt về tốc độ thực thi giữa các mô hình trước khi bước sang thực hành lập trình trong Python.

GIỚI THIỆU

➢ Ma trận đối chiếu 4 mô hình tính toán nền tảng

Tiêu chí đối chiếu	Tuần tự (Sequential)	Đồng thời (Concurrency)	Song song (Parallelism)	Bất đồng bộ (Asynchronous)
Đơn vị điều phối	Chỉ thị lệnh (Instruction)	Process / Thread	CPU Core vật lý	Coroutine / Task
Yêu cầu phần cứng	≥ 1 Core bất kỳ	≥ 1 Core (Time-slicing)	Đa nhân (Multi-core > 1)	≥ 1 Core (Single-thread)
Khi gặp I/O chờ	Nghẽn hoàn toàn (Block)	HĐH chuyển ngữ cảnh	Core khác vẫn chạy	Tạm nhường quyền (Yield)
Phù hợp bài toán	Tuần tự, đơn giản	I/O blocking đa luồng	CPU-bound nặng	I/O-bound nhiều kết nối